

PART-1:

Describe how you built your N-gram models

I started by reading and tokenizing every sentence in the training file. Then, I created a vocabulary by keeping only tokens that occurred 3 or more times, and replaced everything else with <unk>. After that, I added <STOP> to the end of each sentence, and <START> where needed.

- For the unigram model, I simply counted the frequency of each token using FreqDist.
- For the bigram model, I added <START> at the beginning and used nltk.ConditionalFreqDist() to keep track of how often a word follows another.
- For the trigram model, I added two <START> tokens and tracked how often a word appears given the previous two.

All three models were built from the training set only, as required.

Describe how you computed perplexity without any smoothing.

To compute perplexity (no smoothing), I followed this formula:

$$\text{Perplexity} = e^{-1/N \sum_{i=1}^N \log P(w_i | \text{context})}$$

In simple words:

- I looped through every test sentence.
- For each n-gram (unigram, bigram, trigram), I calculated the log probability of the word given its context.
- If any probability was 0, I returned infinity (inf) since $\log(0)$ is undefined.

- Otherwise, I added the log prob and used the formula to convert back from log-space.

This was done using my custom perplexity() function.

What's the potential downside of Laplace smoothing? Back up your claim with empirical evidence.

The biggest problem with Laplace smoothing is that it adds fake counts to everything, even to unseen or rare words. This makes the probabilities too flat every word is treated as if it's a little more common than it really is.

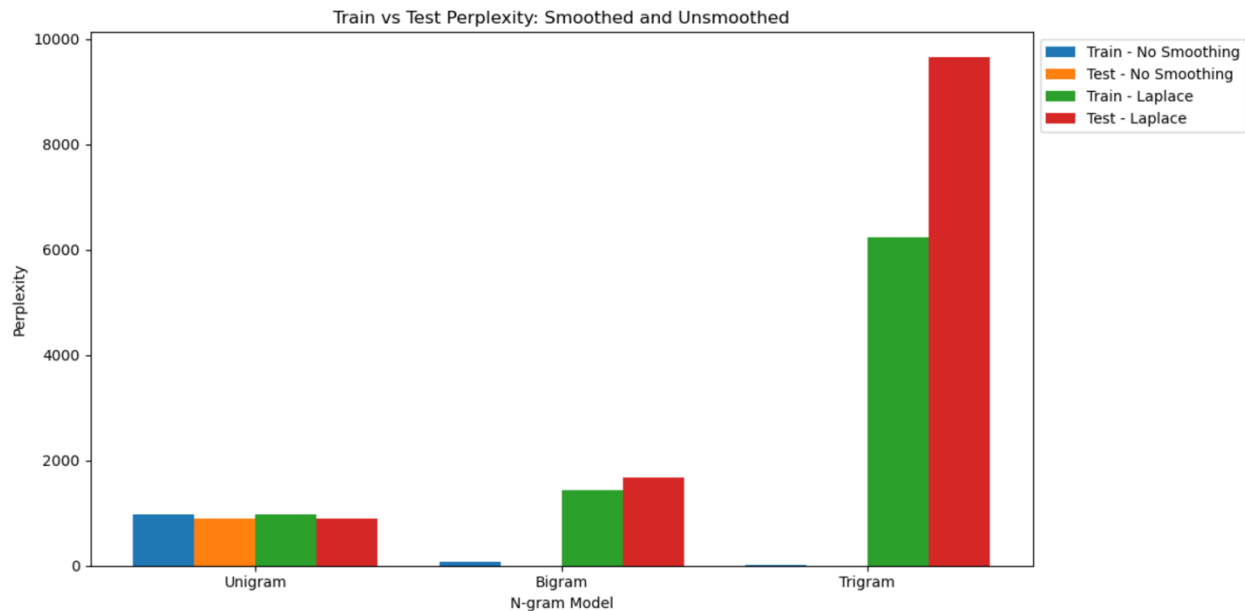
For example, in my results:

- Trigram (test set) with Laplace smoothing gave a perplexity of 9649.6, which is way worse than the unsmoothed trigram (train) with only 7.87.
- It even performs worse than unigrams in some cases, which shouldn't happen ideally.

So Laplace helps when we need to avoid 0 probabilities, but it can hurt performance for higher-order models with sparse data.

Report the unsmoothed and Laplace-smoothed perplexity scores. Briefly discuss.

Perplexity Comparison Table:					
	Model	Train - No Smoothing	Test - No Smoothing	Train - Laplace	Test - Laplace
0	Unigram	976.5437	896.4995	977.5079	898.5562
1	Bigram	77.0735	inf	1442.3087	1665.3873
2	Trigram	7.8730	inf	6244.4249	9649.6073



Discussion:

- The no-smoothing bigram and trigram models give excellent results on the train set, but completely fail (inf) on the test set because of unseen n-grams.
- Laplace helps fix that and gives finite perplexity, but the scores are much worse overall.
- Unigrams remain the most stable across both settings.

PART 2:

Describe how you went through the n-gram models to generate a text.

I first merged both training and test datasets and built new vocab and models from that. Then:

- For bigram, I started with <START>, and for trigram, I used two <START> tokens as seed.
- Then I generated up to 100 words or until <STOP> was seen.
- I tried 3 strategies:
 - Greedy: Always pick the most frequent next word

- Random Sampling: Pick randomly based on probability
- Top-p Sampling (p=0.9): Pick from the smallest group of words whose cumulative probability is ≥ 0.9

The sentence generation was done using a custom `generate_sentence()` function I wrote — no built-in generation tools were used.

Generated Sentences

a. Greedy

- Bigram:
"The <unk> , the <unk> , the <unk> , ..." (repeats <unk>)
- Trigram:
"The <unk> <unk> , <unk> , <unk> , ..."

b. Random Sampling (3)

- Bigram:
 1. The A <unk> million a turbulent area and the planet .
 2. The <unk> 's chief says the turmoil better at the public diplomacy , there has penned an inquiry by the prosecution .
 3. The Its investments are an eviction process .
- Trigram:
 1. The Bernard Kouchner said in a rundown between third base .
 2. The Germany 's legendary Strip .
 3. The In Manchester a teenager , " the China Daily his country 's human rights organizations are facing tough decisions about some banks .

c. Top-p Sampling (p=0.9, 3 each)

- Bigram:
 1. Posted by a mere \$ <unk> and <unk> .
 2. Everyone wants to set by Thursday from the owner of record of late on its progress there is \$ 161 million .
 3. Figures from the hospital have never seen before the gold futures contract to stay .
- Trigram:
 1. It is not anti-competitive .

2. <unk> Group Limited , " <unk> " -- the rain .
3. It is one thing , the Economist Intelligence Unit forecasts a flat in Clapham , south London , the report , what about fitness " and I think all sides reached a conclusion .

Final Reflection

- I now understand how language models use word history to predict the next word and how smoothing helps with unseen data. Also learned how generation strategies like greedy vs top-p lead to very different sentence structures.
- I'd say medium. The N-gram part was okay, but computing perplexity properly and handling infinity cases needed careful thinking. Top-p sampling also took time to get right.
- Getting bigram/trigram to not crash on unseen sequences. Getting correct log probabilities and smoothing math. Making sure <START> and <STOP> tokens were handled properly. Writing sentence generation that actually stops when it should.
- Probably wrap the code into a proper NGramModel class so everything is modular. Would visualize word predictions to better debug.